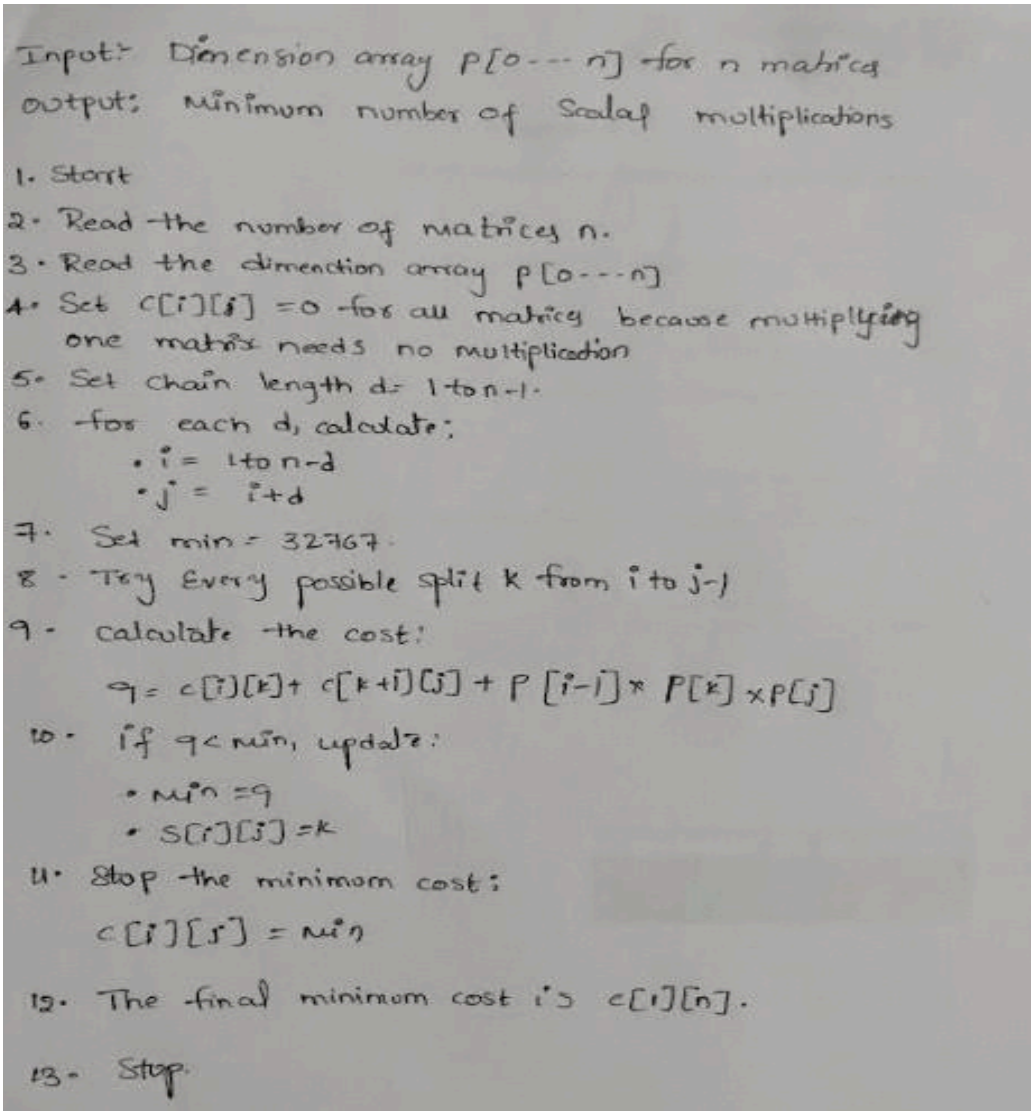
Practical: 6

Aim: Implementation of chain matrix multiplication using dynamic programming.

Algorithm:



PROGRAM

```
#include <iostream>
#include <climits>
using namespace std;

voidprintTable(intm[][100], int n)
{
   cout<<"\nDPTable(Minimum Costs):\n";

   for(inti =1; i <n;i++)
   {
      for(intj=1;j<n;j++)
      {
         if (i > j)
            cout << "-\t";
         else
            cout<<m[i][j]<< "\t";
      }
      cout << endl;
   }
}

intmatrixChainOrder(intp[], int n)
{
   int m[100][100];

   for(inti =1; i <n;i++)
   {
      m[i][i] =0;
   }

   for(intL=2;L<n;L++)
   {
      for(inti=1;i<n-L+ 1; i++)
      {
         intj=i+L-1;

         //Setinitialcosttoinfinity
         m[i][j]=INT_MAX;
```

```
            for (int k = i; k < j; k++)
            {
                int q = m[i][k]
                    + m[k + 1][j]
                    + p[i - 1] * p[k] * p[j];

                if (q < m[i][j])
                {
                    m[i][j] = q;
                }
            }
        }
    }

    printTable(m, n);

    return m[1][n - 1];
}

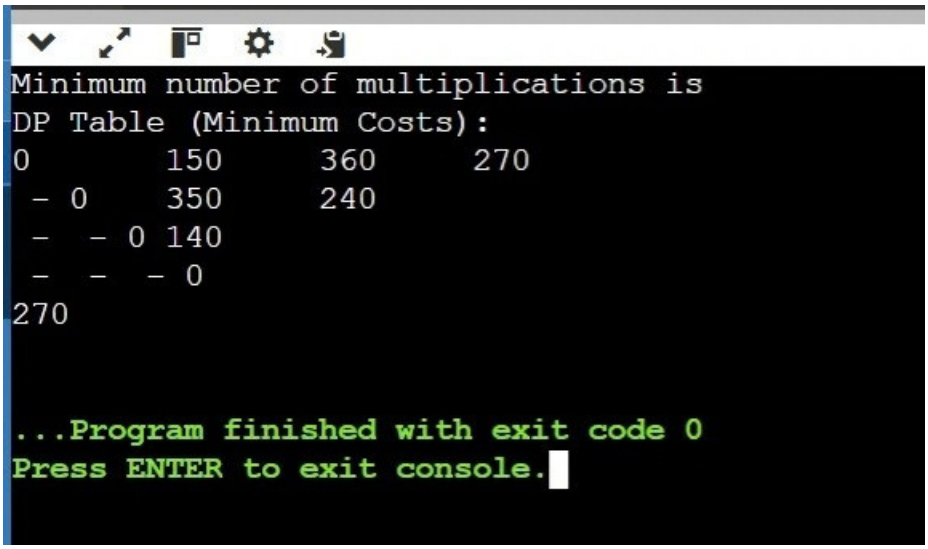
// Main function
int main()
{
    // Dimensions of matrices:
    // A1 = 3 x 5
    // A2 = 5 x 10
    // A3 = 10 x 7
    // A4 = 7 x 2
    int arr[] = {3, 5, 10, 7, 2};


    int n = sizeof(arr) / sizeof(arr[0]);

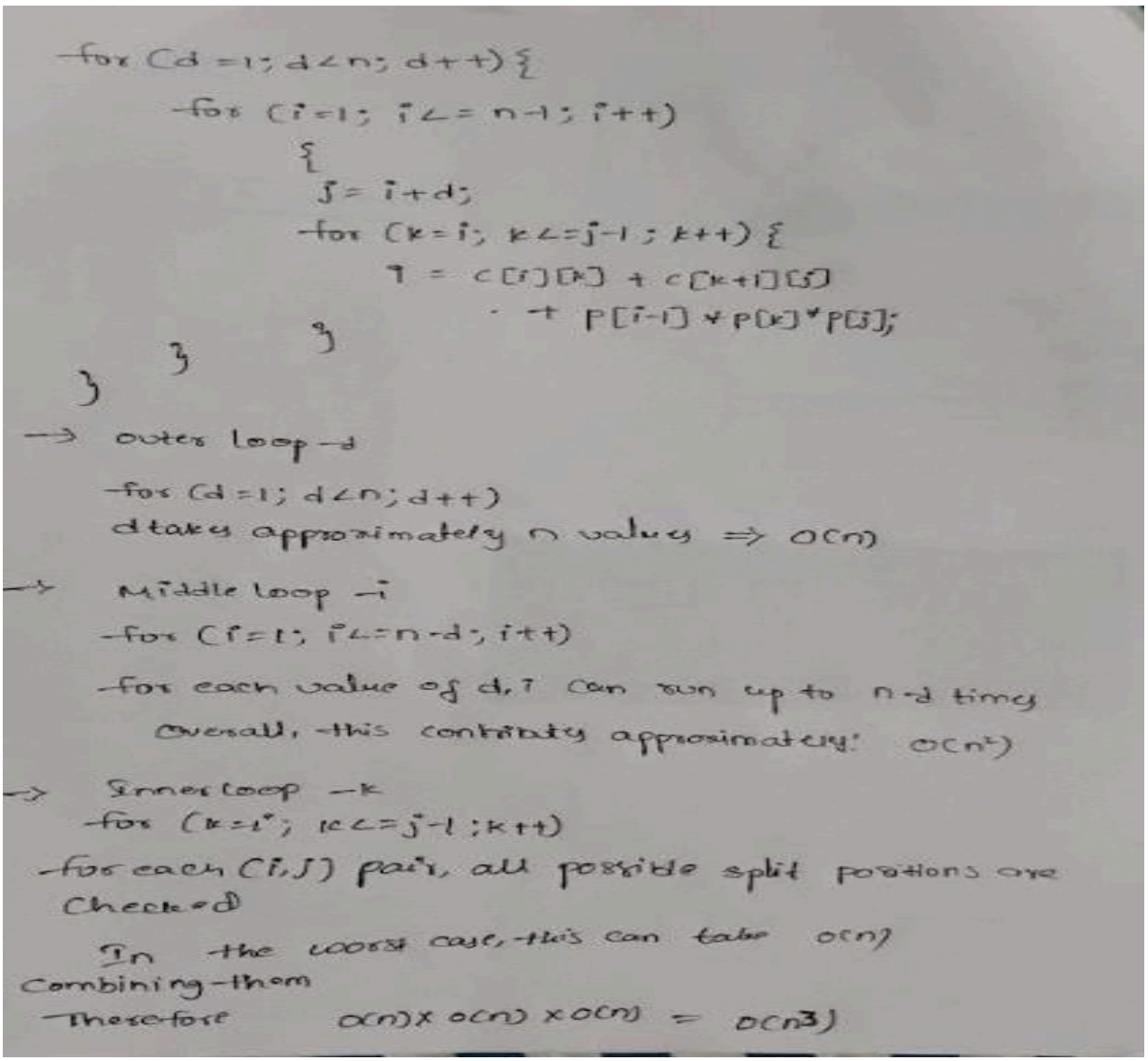
    cout << "Minimum number of
multiplications is: "
         << matrixChainOrder(arr, n);

    return 0;
}
```

OUTPUT:


Minimum number of multiplications is
DP Table (Minimum Costs):
0 150 360 270
- 0 350 240
- - 0 140
- - - 0
270
...Program finished with exit code 0
Press ENTER to exit console.

Time Complexity:



Conclusion:
Matrix Chain Multiplication using Dynamic Programming finds the optimal order of multiplying matrices with the minimum number of scalar multiplications. It avoids repeated calculations by storing previous results in a DP table.